



重庆大学
CHONGQING UNIVERSITY

专业综合设计课程报告

基于物联网的作业场所粉尘危害监测预警系统设计与实现

教师 李季

姓名 陈骁睿

学号 20225847

班级 强基物理 01

学院 物理学院

2025 年 1 月 14 日

目录

1	需求分析	4
1.1	粉尘传感器模拟 (Client)	4
1.2	作业工人轨迹模拟 (Client)	4
1.3	作业场所粉尘危害监测预警系统 (Server)	4
1.3.1	数据采集管理	4
1.3.2	粉尘危害监控	4
1.3.3	粉尘危害预警	4
1.3.4	累积接尘量监控	4
1.3.5	累积接尘量预警	5
1.3.6	数据统计分析	5
2	系统设计	5
2.1	系统架构	5
2.2	功能模块设计	5
2.2.1	粉尘传感器模拟模块	5
2.2.2	作业工人轨迹模拟模块	5
2.2.3	数据采集管理模块	5
2.2.4	粉尘危害预警模块	5
2.2.5	累积接尘量监控模块	6
2.2.6	累积接尘量预警模块	6
2.2.7	数据统计分析模块	6
2.3	核心功能设计思路	6
2.3.1	传感器采集数据生成	6
2.3.2	工人移动轨迹数据生成	6
2.3.3	TcpServer 数据接收与存储	6
2.3.4	累计接尘量计算	6
3	代码实现	7
3.1	前端	7
3.2	后端	8
3.2.1	SensorClient.java	8
3.2.2	TcpServer.java	8
3.2.3	WorkerClient.java	9
3.2.4	Database.java	9
3.2.5	App.java	10
4	功能展示	11

5 总结

13

1 需求分析

典型作业场所（煤矿、建筑工地、生产企业等）会产生大量粉尘，其中呼吸性粉尘对作业工人的危害极大。随着物联网技术的发展，粉尘的检测技术越来越成熟，典型作业场所粉尘精细化检测成为可能。为此，基于粉尘传感器采集的环境粉尘浓度，设计并开发一套基于物联网的作业场所粉尘危害监测预警系统，实现以下主要功能：

1.1 粉尘传感器模拟（Client）

实现作业场所固定位置点的粉尘传感器相关数据的随机产生。按照 5 分钟频率随机产生的监测参数包括：粉尘浓度 mg/m^3 、环境温度 $^{\circ}\text{C}$ 、环境湿度和其他信息（根据需要自由发挥）；同时，实时上传固定位置点的视频图像。

1.2 作业工人轨迹模拟（Client）

在作业场所，作业工人在粉尘环境下进行作业，模拟作业工人的移动轨迹，同时可上传作业视频。

1.3 作业场所粉尘危害监测预警系统（Server）

在完成 Client 的数据的采集与存储，实现作业场所粉尘危害监控、作业工人累积接尘量监控、作业工人接尘量预警、作业场所粉尘预警等功能。

1.3.1 数据采集管理

实现粉尘传感器、作业工人轨迹的采集与管理。

1.3.2 粉尘危害监控

针对作业场所固定位置点，实时查看所采集的历史相关参数，并进行分析；实时查看现场视频与图像。

1.3.3 粉尘危害预警

针对作业场所固定位置点，对出现的粉尘危害超限情况进行预警。

1.3.4 累积接尘量监控

针对作业工人，计算作业工人累积接尘量；并对历史接尘量进行可视化展现。

1.3.5 累积接尘量预警

针对作业工人，设计预测方法，对累积接尘量进行预测；并对作业工人累积接尘量超限情况进行预警。

1.3.6 数据统计分析

对数据进行多维度查询与统计分析。

2 系统设计

2.1 系统架构

系统采用 B/S 架构，分为前端展示层、后端服务层和数据库层。前端展示层使用 Vue.js 框架，后端服务层使用 Java 语言，数据库采用 MySQL。

2.2 功能模块设计

2.2.1 粉尘传感器模拟模块

该模块通过 SensorClient 类实现，模拟粉尘传感器采集的数据，包括粉尘浓度、环境温度、环境湿度等，并通过 Socket 连接将数据发送到服务器。同时，使用 WebcamPanel 类实现视频图像的采集和上传。

2.2.2 作业工人轨迹模拟模块

该模块通过 WorkerClient 类实现，模拟作业工人的移动轨迹，并将轨迹数据发送到服务器。轨迹数据包括工人的位置坐标、时间等信息。

2.2.3 数据采集管理模块

该模块通过 TcpServer 类和 ServerThread 类实现，负责接收来自粉尘传感器和作业工人轨迹模拟模块的数据，并存储到数据库中。使用 Database 类进行数据库操作，包括数据的插入、查询等。

2.2.4 粉尘危害预警模块

该模块通过对数据库中粉尘浓度数据的实时监测，当粉尘浓度超过设定阈值时，触发预警机制，通过前端页面标红警告相关人员。

2.2.5 累积接尘量监控模块

该模块通过计算作业工人在不同时间点的粉尘暴露浓度，累计得到总接尘量，并将数据存储到数据库中。前端通过图表展示工人累积接尘量的变化趋势。

2.2.6 累积接尘量预警模块

该模块通过对工人累积接尘量的预测分析，计算安全周期，提醒工人注意防护或采取相应措施。

2.2.7 数据统计分析模块

该模块提供多维度的数据查询和统计分析功能，用户可以根据不同的条件，如时间范围、传感器编号等，查询相关的粉尘浓度数据、温度数据、湿度数据等，并生成统计报表。

2.3 核心功能设计思路

2.3.1 传感器采集数据生成

假设传感器是二维平面上的点，可以设置传感器的编号、坐标等属性。通过 `SensorData` 类随机生成传感器数据，包括日期、时间、粉尘浓度、温度、湿度和风力等信息。通过 `SensorClient` 类模拟传感器数据的生成，包括历史数据和实时数据，通过 `Socket` 连接将数据发送到服务器。

2.3.2 工人移动轨迹数据生成

假设工人的运动轨迹是二维平面上的有序点，可以设置工人的姓名、工作天数等属性。通过 `PathVertex` 类生成工人轨迹点，包括日期、时间、X 坐标和 Y 坐标等信息。通过 `WorkerPath` 类生成工人的轨迹数据，包括一天的轨迹点列表。`WorkerClient` 类通过 `Socket` 连接将数据发送到服务器。

2.3.3 TcpServer 数据接收与存储

通过继承 `java.net.ServerSocket` 类实现 `TcpServer` 类，监听客户端连接，接收并处理客户端上传的数据。通过继承 `java.lang.Thread` 类实现 `ServerThread` 类，处理单个客户端的连接和数据交互。通过 `Database` 类实现数据的插入、查询等操作，将传感器数据和工人数据存储到数据库中。

2.3.4 累计接尘量计算

分别计算工人与四个传感器之间的欧几里得距离（平面内）

$$dis = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

以距离为权重计算每小时的平均粉尘量

$$\text{averageDensity} = 0.02 \times \frac{\left(\frac{\text{den}_1}{\text{dis}_1+1} + \frac{\text{den}_2}{\text{dis}_2+1} + \frac{\text{den}_3}{\text{dis}_3+1} + \frac{\text{den}_4}{\text{dis}_4+1} \right)}{4}$$

其中 den 是粉尘浓度，dis 是距离

3 代码实现

3.1 前端

前端使用了开源框架，实现了粉尘监测系统的前端页面展示和交互功能。前端主要包括以下几个组件：

App.vue

功能：作为应用入口，通过顶部菜单切换不同功能页面，并将当前菜单项的值存放在 ‘this.store.state.currMenuItemKey’ 里。

sensor.vue

功能：提供查询和动态获取指定日期、指定传感器序号的粉尘、温度、湿度、风力等数据，并以表格形式展示。

worker.vue

功能：获取工人的工作轨迹数据，支持在模态对话框中对坐标点进行绘制，可视化工人路径。

real-time-monitor.vue

功能：实时监测指定序号的传感器数据，并支持调用摄像头进行视频查看或截图保存，将数据和图表一并展示。

accumuted-dust.vue

功能：根据日期查询工人的累计接尘信息，并通过折线图展示每个时间点对应的累积粉尘数值。

analysis-dust-volume.vue

功能：通过选择起止日期，从后端获取预测的接尘量信息，包含安全周期等统计指标，并将时序粉尘数值可视化成折线图。

multidivision-analysis.vue

功能：支持多维度条件筛选（温度、湿度、风力、粉尘等）和排序，发送给后端后获取并显示相应的多维度分析结果表格。

3.2 后端

3.2.1 SensorClient.java

SensorData

功能：表示传感器数据，包含日期、时间、粉尘浓度、温度、湿度和风力等信息。

实现方法：

构造函数：初始化传感器数据对象，设置日期、时间、粉尘浓度、温度、湿度和风力等属性。

getter 方法：提供获取各个属性值的方法，如 'getDensity()' 获取粉尘浓度，'getDate()' 获取日期等。

toString 方法：重写 'toString' 方法，方便打印传感器数据信息。

SensorClient

功能：传感器客户端，用于生成传感器数据并上传至服务器。

实现方法：

main 方法：程序入口，创建传感器客户端对象，连接服务器，并上传传感器历史数据和实时数据。

构造函数：初始化传感器客户端，设置传感器编号、坐标，并生成指定天数的历史数据。

uploadData 方法：将传感器客户端对象（包含历史数据）上传至服务器，通过对象输出流将对象序列化后发送。

uploadRealTimeData 方法：周期性生成传感器实时数据并上传至服务器，使用 'ScheduledExecutorService' 定时执行数据生成和上传任务。geneSensors 方法：随机生成指定天数的传感器历史数据，数据包括日期、时间、随机生成的粉尘浓度、温度、湿度和风力等。openSensorCamera 方法：打开摄像头，使用 'Webcam' 库显示摄像头画面。

3.2.2 TcpServer.java

TcpServer

功能：TCP 服务器，监听客户端连接，接收并处理客户端上传的数据。

实现方法：

main 方法：程序入口，创建服务器套接字，监听 12345 端口，等待客户端连接，并为每个连接的客户端创建一个新的 'ServerThread' 线程进行处理。

ServerThread

功能：服务器线程，处理单个客户端的连接和数据交互。

实现方法：

构造函数：初始化服务器线程，设置客户端套接字，并创建数据库对象。run 方法：线程运行时执行的方法，通过对象输入流接收客户端上传的数据对象，根据对象类型调用相应的数据库存储方法。

storeToDatabase 方法（多个重载版本）：将客户端上传的数据存储到数据库中，包括传感器历史数据、实时数据和工人客户端数据。

openCamera 方法：根据传感器编号打开对应的摄像头。

3.2.3 WorkerClient.java

PathVertex

功能：表示轨迹点，包含日期、时间、X 坐标和 Y 坐标等信息。

实现方法：

构造函数：初始化轨迹点对象，设置日期、时间、X 坐标和 Y 坐标等属性。

toString 方法：重写 'toString' 方法，方便打印轨迹点信息。

WorkerPath

功能：表示工人一天的轨迹，由多个轨迹点构成。

实现方法：

构造函数：初始化工人轨迹对象，设置日期，并生成一天的轨迹数据。

Generate_Path_Data 方法：生成一天的轨迹数据，包含 48 个轨迹点，每个点的时间间隔为 10 分钟。

generateCoordinate 方法：辅助方法，用于生成新的坐标，坐标生成有范围限制。

getAllVertex 方法：获取轨迹点列表。

toString 方法：重写 'toString' 方法，方便打印工人轨迹信息。

WorkerClient 功能：工人客户端，用于生成工人在指定天数内的轨迹数据，并上传至服务器。

实现方法：

main 方法：程序入口，创建工人客户端对象，连接服务器，并上传工人轨迹数据。

构造函数：初始化工人客户端，设置工人姓名、工作天数，并生成指定天数的工人轨迹数据。

generatePath 方法：生成工人在指定天数内的轨迹数据，每天生成一条轨迹。

uploadWorkerPaths 方法：将工人客户端对象（包含轨迹数据）上传至服务器，通过对象输出流将对象序列化后发送。

getPathsArray 方法：获取工人路径列表。

drawPathOfDate 方法：获取并绘制指定日期的工人轨迹，但具体绘制方法未实现。

toString 方法：重写 'toString' 方法，方便打印工人轨迹信息。

3.2.4 Database.java

Database

功能：该类负责与数据库进行交互，实现传感器数据和工人数据的插入、查询、清空等操作，为应用提供数据存储与检索功能。

实现方法：

`accessDB` 方法：加载数据库驱动，创建并返回数据库连接，连接参数包括数据库 URL、用户名和密码等，为后续的数据库操作提供连接支持。

`clearSensor` 方法：清空指定传感器编号的传感器表中的数据。

`clearWorker` 方法：清空工人表中的所有数据。

`insertSensor` 方法：插入传感器数据，根据传入的传感器编号和 `SensorData` 对象，将数据插入到对应的传感器表中，包括日期、时间、粉尘浓度、温度、湿度和风力等信息。

`insertWorker` 方法：插入工人数据，将 `PathVertex` 对象中的工人位置信息（日期、时间、X 坐标、Y 坐标）插入到工人表中。

`getSensor` 方法：查询指定传感器编号的所有数据，返回包含传感器数据的 `ArrayList`。

`getWorker` 方法：查询所有工人数据，返回包含工人位置信息的 `ArrayList`。

`getSensorByDate` 方法：根据传感器编号和日期查询传感器数据，返回特定日期的传感器数据列表。

`getWorkerByDate` 方法：根据日期查询工人数据，返回该日期的工人位置信息列表。

`workerDust` 方法：查询用于计算接尘量的数据，包括工人位置和同时刻各传感器的浓度，返回一个 `ResultSet`。

`queryAndSort` 方法：实现多维统计分析查询，根据传入的条件（传感器编号、粉尘浓度、温度、湿度、风力、比较方式）查询传感器数据，并返回查询结果列表。

`queryAndSortByDate` 方法：在特定日期范围内进行多维统计分析查询，除了基本查询条件外，还包括开始和结束日期、排序键和排序方式，返回符合条件的传感器数据列表。

3.2.5 App.java

App

功能：这是粉尘监测系统的核心应用类，负责搭建 HTTP 服务器，处理来自不同客户端的请求，并将数据库中的数据以 JSON 格式响应给客户端，实现传感器数据、工人数据等的查询与分析功能。

实现方法：

`main` 方法：程序入口，创建 HTTP 服务器，监听 8001 端口，并为不同的请求路径创建对应的上下文及请求处理器。

`SensorHandler` 类：处理传感器数据查询请求，解析请求体中的日期和传感器编号，从数据库获取对应日期的传感器数据，将其封装成 JSON 格式响应给客户端。

`WorkerHandler` 类：处理工人数据查询请求，根据请求体中的日期，查询数据库中该日期的工人位置数据，并以 JSON 格式返回。

`SensorRealTimeHandler` 类：用于实时监测传感器数据，根据请求体中的传感器编号，从数据库获取该传感器的最新数据，并以 JSON 格式返回给客户端。

WorkerDustHandler 类：查询工人某天吸入的粉尘量，通过调用 ‘getWorkerDust’ 方法计算工人在特定日期的累积接尘量，并将结果封装成 JSON 格式响应。

DustPretHandler 类：进行接尘量预测，根据请求的开始和结束日期，计算工作周期、累积接尘以及安全周期等信息，并返回预测的接尘量数据。

MultiAnalyHandler 类：实现多维统计分析（查询）功能，根据请求中的粉尘浓度、温度、湿度、风力等条件以及排序方式，从数据库查询符合条件的传感器数据，并以 JSON 格式返回。

MultiSortHandler 类：用于多维统计分析（筛选），根据请求的开始和结束日期、排序方法及排序顺序，从数据库查询并筛选传感器数据，返回给客户端。

getWorkerDust 方法：计算工人某天吸入的粉尘量，通过查询数据库中工人位置与传感器浓度数据，结合工人与传感器的距离等因素，计算工人在不同时间段的平均粉尘暴露量，并累加得到全天的累积接尘量。

totalDust 方法：计算在给定的开始和结束日期之间的累积接尘量，考虑工作日与周末的不同情况，模拟工人在工作日积累粉尘、周末自净化的过程，返回累积接尘量列表。

Preceed 方法：HTTP 请求的预处理方法，设置响应头以处理跨域问题，指定返回数据格式为 JSON，并发送响应头。

4 功能展示

传感器数据收集

日期	时间	粉尘浓度	温度	湿度	风力
2024-01-01	09:05:00	5.18	19.9	49	7
2024-01-01	09:10:00	5.54	19.7	50	7
2024-01-01	09:15:00	6.43	18.9	50	8
2024-01-01	09:20:00	6.95	18.3	51	7
2024-01-01	09:25:00	6.92	18.6	52	7
2024-01-01	09:30:00	7.5	18.1	52	8
2024-01-01	09:35:00	6.97	18.7	52	8
2024-01-01	09:40:00	7.34	18.7	51	9
2024-01-01	09:45:00	7.6	18.0	51	9
2024-01-01	09:50:00	7.56	17.9	50	8

图 1: 传感器数据收集

工人路径收集

传感器实时检测

查看现场视频（模拟）

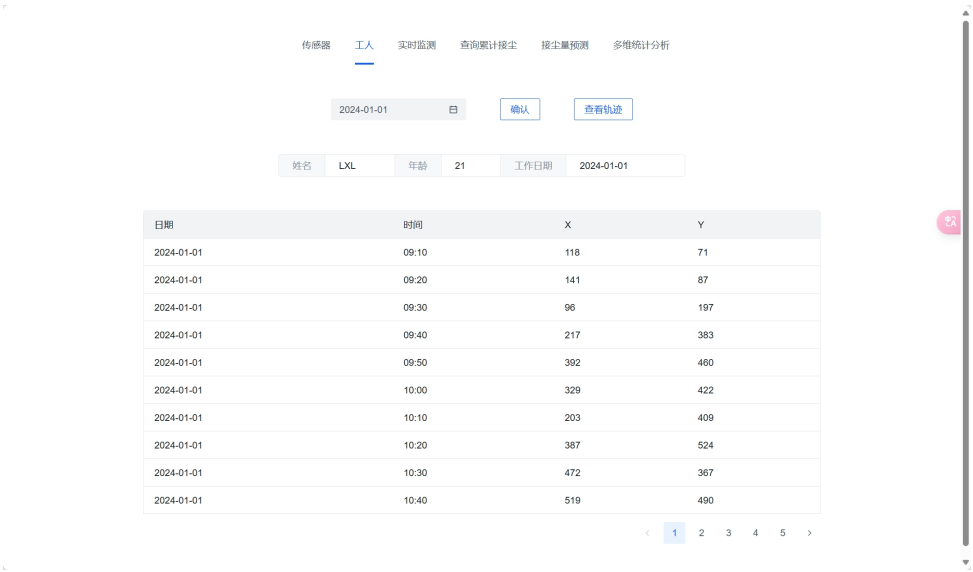


图 2: 工人路径收集

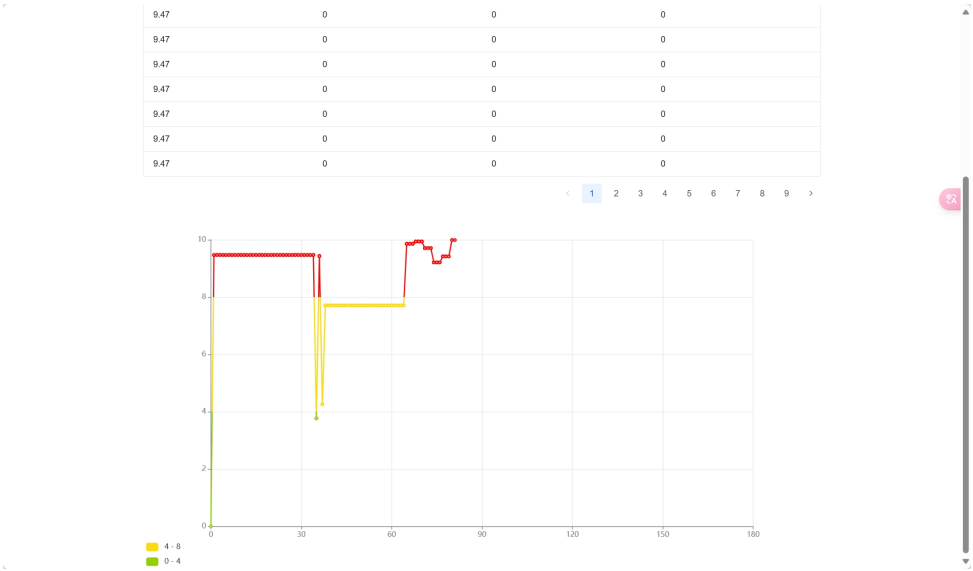


图 3: 传感器实时检测

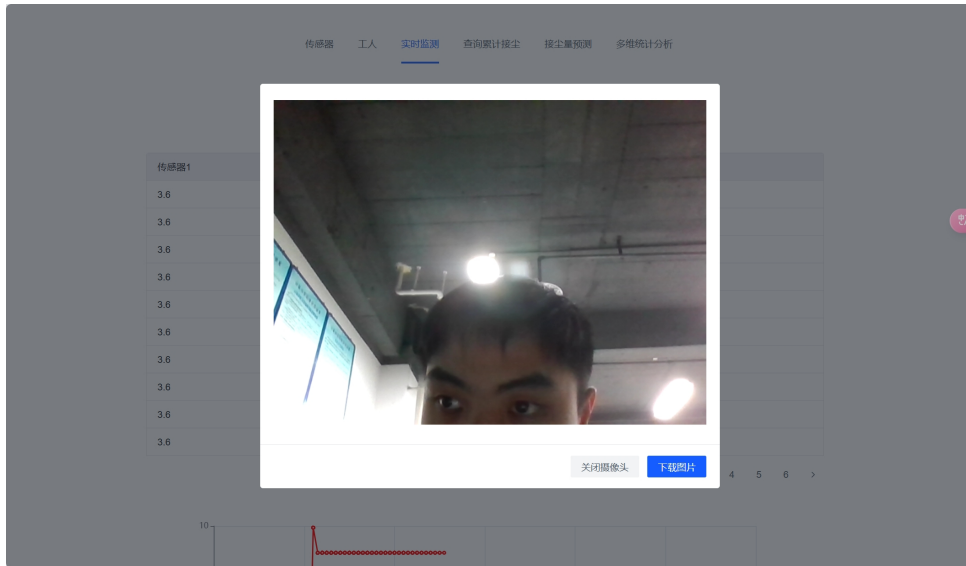


图 4: 传感器实时检测

预测累计接尘量，计算安全周期。

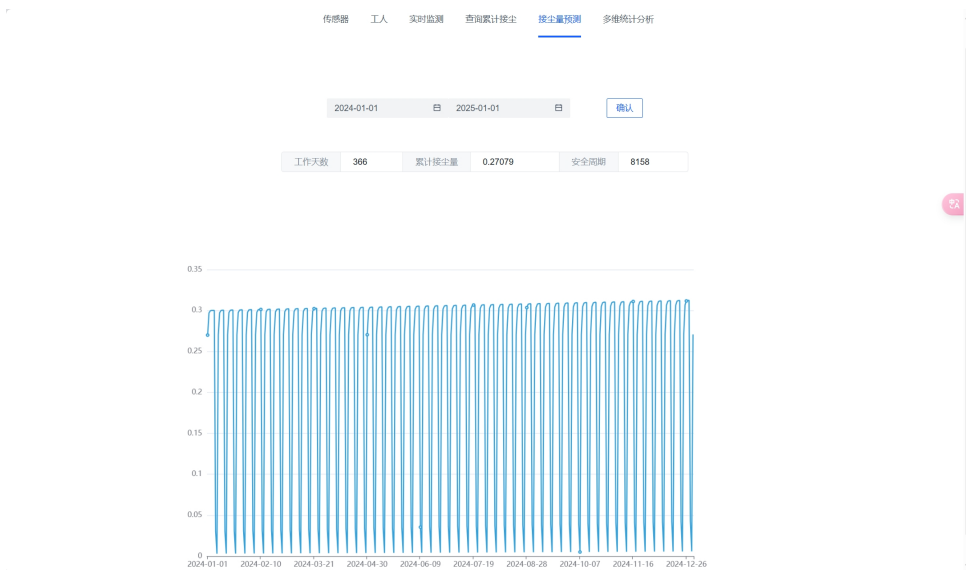


图 5: 统计累计接尘量

多维统计分析

5 总结

本文设计并实现了一套基于物联网的作业场所粉尘危害监测预警系统，实现了粉尘传感器模拟、作业工人轨迹模拟、数据采集管理、粉尘危害监控、粉尘危害预警、累积接尘量监控、累积接尘量预警、数据统计分析等功能。系统采用 B/S 架构，前端使用 Vue.js 框架，后端使用 Java 语言，数据库采用 MySQL。通过前后端的协作，实现了系

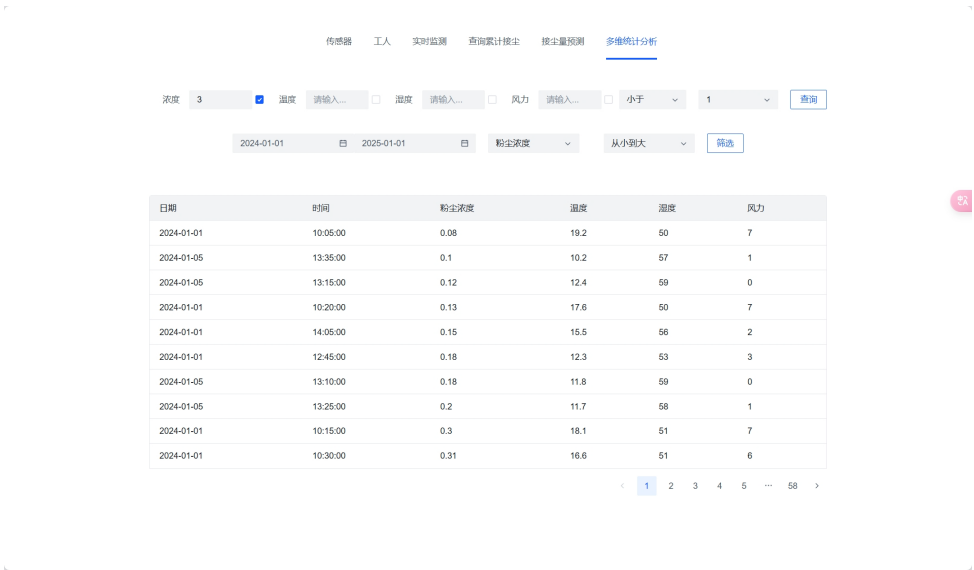


图 6: 多维统计分析

统的各项功能，并通过功能展示，展示了系统的运行效果。

通过这个项目，学习到了物联网技术的应用，了解了粉尘危害监测预警系统的设计与实现，提高了系统设计和开发的能力。在今后的工作中，将继续深入学习物联网技术，不断提升自己的技术水平，为实现更多的智能化应用做出贡献。